

Exercise 2: Error Handling

Aim

To implement error handling in MySQL stored procedures using transactions, exception handlers, and rollback operations to ensure data integrity.

```
CREATE DATABASE BANKACCOUNT;
```

```
USE BANKACCOUNT;
```

```
CREATE TABLE Account(  
    AccountID INT PRIMARY KEY,  
    CustomerID INT,  
    Balance DECIMAL(10,2)  
);
```

```
INSERT INTO Account VALUES  
(1,101,50000),  
(2,102,30000),  
(3,103,15000);
```

```
CREATE TABLE Employee(  
    EmployeeID INT PRIMARY KEY,  
    EmployeeName VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

```
INSERT INTO Employee VALUES  
(201,'Ravi',50000),  
(202,'Anita',60000),  
(203,'John',45000);
```

--**Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

DELIMITER \$\$

CREATE PROCEDURE SafeTransferFunds(

IN FromAcc INT,

IN ToAcc INT,

IN Amount DECIMAL(10,2)

)

BEGIN

DECLARE Bal DECIMAL(10,2);

DECLARE EXIT HANDLER FOR SQLEXCEPTION

BEGIN

ROLLBACK;

SELECT 'Transaction Failed' AS Message;

END;

START TRANSACTION;

SELECT Balance

INTO Bal

FROM Account

WHERE AccountID=FromAcc;

IF Bal<Amount THEN

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Insufficient Funds';
```

```
END IF;
```

```
UPDATE Account
```

```
SET Balance=Balance-Amount
```

```
WHERE AccountID=FromAcc;
```

```
UPDATE Account
```

```
SET Balance=Balance+Amount
```

```
WHERE AccountID=ToAcc;
```

```
COMMIT;
```

```
SELECT 'Transfer Successful' AS Message;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL SafeTransferFunds(1,2,5000);
```

	Message
▶	Transaction Failed

--**Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

DELIMITER \$\$

CREATE PROCEDURE UpdateSalary(

IN EmpID INT,

IN PercentIncrease DECIMAL(5,2)

)

BEGIN

IF EXISTS

(

SELECT *

FROM Employee

WHERE EmployeeID=EmpID

)

THEN

UPDATE Employee

SET Salary=Salary+(Salary*PercentIncrease/100)

WHERE EmployeeID=EmpID;

SELECT 'Salary Updated Successfully' AS Message;

ELSE

```
SELECT 'Employee ID Not Found' AS Message;
```

```
END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL UpdateSalary(999,10);
```

	Message
▶	Employee ID Not Found

```
CALL UpdateSalary(201,20);
```

	Message
▶	Salary Updated Successfully

--**Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
DELIMITER $$
```

```
CREATE PROCEDURE AddNewCustomer(
```

```
IN CID INT,
```

```
IN CName VARCHAR(50),
```

```
IN CAge INT,
```

IN CBalance DECIMAL(10,2),

IN Interest DECIMAL(5,2)

)

BEGIN

IF EXISTS

(

SELECT *

FROM Customer

WHERE CustomerID=CID

)

THEN

SELECT 'Customer ID Already Exists' AS Message;

ELSE

INSERT INTO Customer

VALUES

(CID,CName,CAge,CBalance,Interest,FALSE);

SELECT 'Customer Added Successfully' AS Message;

END IF;

END\$\$

DELIMITER ;

CALL AddNewCustomer(106,'Karan',30,12000,9.5);

	Message
►	Customer Added Successfully